

Agents: A Two-Day Guide

From zero to designing a multi-agent pipeline — written for the IBM Bob 2.0 build

21 September 2026 · Prepared for team minahiru

How to use this guide

This takes two days and assumes you know how to code but have never built anything agentic. By the end you will be able to design a multi-agent pipeline and defend the design choices to a judge.

The guide is deliberately narrow. It teaches the concepts you need for the IBM Bob 2.0 build and skips the rest of the field — no model training, no RAG architecture, no framework tour. Breadth is what makes agent material feel overwhelming, and you do not have time for breadth.

BLOCK	HOURS	COVERS
Day 1 morning	3	LLM basics, the agent loop, tools
Day 1 afternoon	3	Context, prompting, the three modes
Day 1 evening	1.5	Hands-on exercise
Day 2 morning	3	Subagents, orchestration patterns, skills
Day 2 afternoon	3	MCP, failure modes, measurement
Day 2 evening	1.5	Design your own pipeline

The one rule: do the two exercises. Reading about agents produces a comfortable feeling of understanding that collapses the moment you try to build one. The exercises are where the learning actually happens. If you are short on time, cut reading, not doing.

Part 0 — Just enough about language models

A language model predicts the next chunk of text, over and over. Everything else you have heard about them is built on top of that one behaviour.

Four facts do all the work in this guide.

Text is split into tokens. A token is roughly 3–4 characters — `understanding` might be two tokens, `the` is one. The model reads and writes tokens, not words. This matters only because tokens are the unit everything is measured and billed in.

The context window is the model's entire world. Every request sends the model a block of text — your instructions, the conversation so far, any file contents — and that block has a size limit. Nothing outside it exists. A model cannot "look at your repository"; something must read files and paste the text in.

The model is stateless between calls. It does not remember your last message. The illusion of memory comes from the whole conversation being re-sent every time. This is the single most useful thing to understand, because it explains nearly every strange agent behaviour you will see.

Output is probabilistic. The same input can give different output. This is why agents need verification rather than trust, a point the whole of Day 2 returns to.

A consequence worth sitting with: when an agent "forgets" an instruction you gave twenty minutes ago, nothing broke. The instruction either fell out of the context window or got buried under so much other text that it stopped carrying weight. The fix is never to repeat yourself louder — it is to manage what goes into the window. That is the subject of a later section, and it is the skill that separates people who find agents frustrating from people who find them useful.

What actually makes something an agent

An agent is a language model running in a loop, with the ability to take actions and see their results. That is the whole definition.

A chatbot takes your text and returns text. One turn, one answer, done. An agent is given a goal and then repeats a cycle until the goal is met: decide what to do, do it, look at what happened, decide again.



The loop is what creates every property people find impressive. An agent can recover from an error because it sees the error and tries again. It can explore because it reads one file, learns a name, and uses that name to find the next file. It can do multi-step work because each step's output becomes the next step's input.

The loop is also what creates every property people find alarming. It can take a wrong turn early and spend twenty steps building on a false premise. It can get stuck repeating a failing action. It can do something destructive quickly, because nothing pauses between steps unless you make it pause.

The practical takeaway for your build: when you design an agentic system, you are not writing prompts. You are designing what the loop can see, what it can do, and when it must stop and ask. Those three decisions determine whether the thing works.

Tools — how a text predictor learns to act

A tool is a function the model is allowed to call. Tools are the only reason an agent can affect anything outside its own text output.

The mechanism is simpler than it sounds, and it is worth understanding exactly, because almost every agent bug traces back to it.

You give the model a list of available functions, each with a name, a description, and its parameters. When the model wants to use one, it does not run anything — it cannot. It emits a structured request that says *"call `read_file` with path `src/auth.js`"*. Your surrounding program sees that request, actually runs the function, and pastes the result back into the conversation. The model then continues with that result now in its context.

A trace of three steps, asking an agent where authentication happens:

STEP	WHO	WHAT HAPPENS
1	Model	Requests <code>search_code("authenticate")</code>
2	Program	Runs the search, returns 3 file paths
3	Model	Reads the paths, requests <code>read_file("src/auth/session.js")</code>
4	Program	Returns 200 lines of source
5	Model	Answers, citing the real function it just read

Notice what happened at step 5. The answer is grounded in text the model actually saw, not in a guess about what a file called `auth.js` probably contains. **This is the entire reason agents are more reliable than chatbots for codebase work** — not better reasoning, just better evidence.

Two consequences that will matter on Day 2:

- **Tool descriptions are prompts.** The model chooses tools based on how they are described. A vague description produces wrong tool choices, and it looks like a reasoning failure when it is really a

documentation failure.

- **Every tool result consumes context.** Reading a 5,000-line file spends a large slice of the window on something mostly irrelevant. This pressure is what makes subagents useful, which is the whole argument of the section on them.

Context is the scarce resource

The limiting factor in agent work is not intelligence. It is how much relevant material fits in the window and how much junk is competing with it.

Picture the context window as a desk. You can spread out a certain number of papers. An agent that has read forty files has a desk buried in paper, and the instruction you wrote at the start is somewhere at the bottom. The model has not become less capable — the signal has been diluted.

This produces three failures you will meet within an hour of starting:

Drift. Long sessions wander off the original goal. The goal is still technically in the window but is now outweighed by hundreds of lines of file contents.

Contradiction. You correct the agent, it complies, and ten steps later it reverts. Both your original instruction and your correction are in the window, and the model is averaging them.

Degradation on long inputs. Accuracy falls as the window fills, especially for material in the middle. Sometimes called *context rot*. A model with a large window does not use all of it equally well.

The habits that follow

1. **Start fresh for a new task.** A clean session is not wasteful, it is a reset to full signal. Continuing an unrelated 2-hour session is the most common self-inflicted agent problem.
2. **Prefer summaries to raw dumps.** "Which files handle auth, one line each" beats reading six files in full.
3. **Put instructions that must survive in a persistent file.** Most agentic tools support a project instructions file that is re-injected every turn. Anything that matters across the whole session belongs there, not in a chat message.
4. **Give one task at a time.** Bundling five requests in one message means four get partial attention.

The reason this section sits before the one on subagents is that **context isolation is the real argument for subagents**. Once you understand that the desk gets crowded, the idea of handing a messy investigation to a separate agent with its own clean desk stops being an architecture fashion and starts being obviously correct.

Talking to an agent

Write a task the way you would write it for a competent contractor who has never seen your project and cannot ask you follow-up questions. That single framing gets you most of the way.

The failure mode for beginners is under-specification. You know what you want, the request feels clear to you, and the agent produces something adjacent but wrong. Compare:

- Weak:** "Add tests for the auth module."
- Strong:** "Add unit tests for `src/auth/session.js` , covering token expiry, malformed tokens, and the refresh path. Use the existing Jest setup and match the style in `tests/user.test.js` . Do not modify source files. Run the suite and show me the output."

The strong version does four things the weak one does not: names the file, lists the cases, points at an example to imitate, and sets a boundary.

The four elements worth including

ELEMENT	WHY IT MATTERS	EXAMPLE
Context	Stops wrong assumptions	"This is a Node 18 service using Express"
Concrete goal	Defines done	"Every env var the app reads appears in <code>.env.example</code> "
Constraints	Prevents collateral damage	"Do not touch files outside <code>src/config/</code> "
Acceptance criteria	Lets it self-check	" <code>npm start</code> must succeed on a clean clone"

Three techniques that punch above their weight

Ask for a plan first. "Before writing code, list the files you intend to change and why." You catch a misunderstanding in ten seconds instead of reviewing forty minutes of wrong work.

Make it show evidence. "Cite the file and line for each claim." An agent that must point at real lines hallucinates dramatically less, because the citation requirement forces it to actually go and look.

Tell it what to do when stuck. "If you cannot find the config loader, stop and tell me rather than creating one." Without this, agents fill gaps by inventing, and an invented file is much worse than a question.

That last one is the highest-value sentence you can add to any agent task, and nearly everyone omits it.

The three modes

Most agentic tools separate asking, planning, and doing. IBM Bob names these explicitly as **Ask**, **Plan**, and **Agent** mode, and the separation exists to control how much damage a misunderstanding can cause.

MODE	CAN IT CHANGE FILES?	USE IT WHEN
Ask	No	You are building a mental model: "how does auth work here?"
Plan	No	The task is big enough that a wrong approach wastes real time
Agent	Yes	The approach is settled and you want it executed

The rhythm that works is Ask → Plan → Agent, and beginners skip straight to Agent almost every time. That is the expensive mistake. Agent mode on a task you have not thought through produces a large diff built on a wrong assumption, and reviewing that diff costs more than planning would have.

Plan mode deserves specific attention for your build. A plan is a reviewable artifact. You read it, you spot that the agent misunderstood which directory holds the config, you correct one sentence, and only then does anything get written. The correction cost ten seconds instead of an hour.

There is also a documentation benefit that matters for a hackathon: the plan Bob produces is evidence of orchestration. It shows a judge that you designed a process rather than typing requests until something worked.

Bob additionally offers **BobShell**, a CLI that runs the same capabilities in the terminal and produces a self-documenting trace of every action. For your submission this is close to free evidence — run your pipeline through it and the audit trail generates itself.

Day 1 exercise — 90 minutes

Pick a repository you did **not** write. Any mid-sized open-source project works; something in a language you know but a codebase you have never opened.

Work through these in order and write down what happens. The notes matter more than the results.

- **Ask mode, 15 min.** "Explain what this project does and name its entry point. Cite file paths." Then verify one claim yourself by opening the file. Did it tell the truth?
- **Ask mode, 15 min.** "List every environment variable this project reads, with the file and line where each is read." This is one of your Orient subagents in miniature.
- **Plan mode, 20 min.** "Plan the addition of a `/health` endpoint returning uptime and version." Read the plan critically. Find at least one thing you would change.
- **Agent mode, 20 min.** Execute the corrected plan. Review the diff line by line.
- **Break it deliberately, 20 min.** Give a badly under-specified task — "improve the error handling" — and watch what it does. This is the most instructive 20 minutes of the day.

What you should be able to answer afterwards

1. When the agent got something wrong, was it a reasoning failure or did it simply lack the information? (It is usually the second.)
2. Which of your instructions did it follow precisely, and which did it treat as a suggestion?

3. What did the vague task produce, and what would you add to the prompt to prevent it?

If you can answer those three, you have the working intuition that Day 2 builds on. If you cannot, repeat the last exercise on a different repository rather than moving on — the orchestration material assumes this feel for how agents fail.

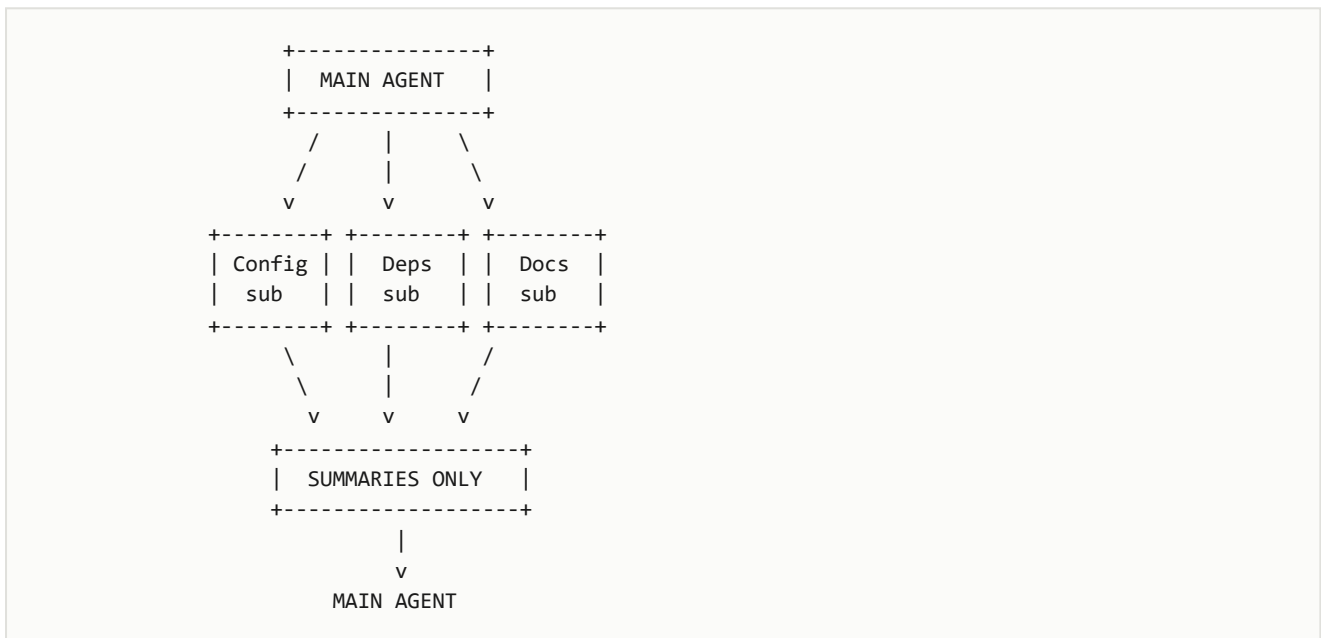
Subagents

A subagent is a separate agent, with its own fresh context window, given a narrow task by a main agent. It works, then returns a summary. Its intermediate mess never enters the main agent's window.

That last sentence is the whole point, and it is not the reason most people give. The common explanation is speed through parallelism. Speed is a real but secondary benefit. The primary benefit is context isolation.

Consider mapping every environment variable in a large repo. Done in your main session, the agent greps, reads twelve config files, follows imports, and dumps perhaps 8,000 tokens of source into the window — of which the useful residue is a list of nine variable names. Your desk is now buried in paper you will never need again.

Give it to a subagent and the main session receives the nine names. The 8,000 tokens of searching happened on someone else's desk.



When subagents make things worse

This matters as much as knowing when they help, and a judge may well ask.

When the subtasks are not independent. If step B needs step A's findings to know what to look for, running them in parallel means B works blind. Sequence them instead.

When the task is small. Spawning a subagent costs a full context setup. For a two-file question it is pure overhead.

When the summary loses what mattered. A subagent returns a compressed answer. If the main agent needed the detail, compression destroyed the thing you wanted. Ask for structured output with specific required fields rather than a free-form summary.

When you need to debug. More agents means more places for a failure to hide. Build the sequential version first, confirm it works, then parallelise the parts that are genuinely independent.

That last point is real advice for your two-day build, not a caveat to skip. A working sequential pipeline you can demo beats an elegant parallel one that breaks on stage.

Orchestration patterns

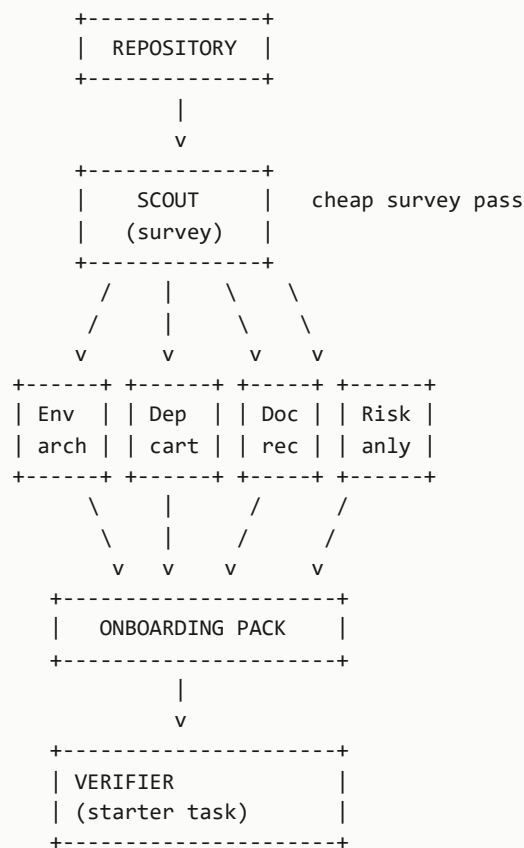
Four arrangements cover almost everything you will build. Knowing their names lets you describe your design to a judge in one sentence instead of five.

PATTERN	SHAPE	USE WHEN
Pipeline	$A \rightarrow B \rightarrow C$	Each stage needs the previous stage's output
Fan-out / fan-in	Parallel, then merge	Investigations are independent
Supervisor	One agent directs others	The next step depends on what was found
Verifier	Producer, then separate checker	Correctness matters more than speed

Fan-out / fan-in is the one your onboarding pipeline is built on. Several subagents investigate different aspects at once, and a final stage merges their findings into one artifact. It works precisely because config extraction, dependency mapping, and documentation reconciliation do not need to know about each other.

The verifier pattern is the one that wins hackathons. A second agent, with no memory of how the first one worked, checks the output against reality. It is powerful because the checker has no stake in the answer being right — it is not defending work it just did. Your "fresh agent completes a starter task using only the generated onboarding pack" step is exactly this, and it converts a claim into a demonstration.

Orient, as a diagram



Read it as three phases: a cheap survey pass that tells the parallel agents where to look, four independent investigations, then assembly and an independent check. The survey matters — without it, four agents each waste their opening moves rediscovering the same project layout.

That design decision is worth stating explicitly in your submission. It shows you thought about cost and duplication, not just about drawing boxes in parallel.

Skills — turning a prompt into an asset

A skill is a reusable, named workflow: instructions you write once and invoke repeatedly, instead of retyping a long prompt and getting slightly different behaviour each time.

The difference is the difference between a shell command you typed and a shell script you saved. Same content, entirely different value — because one is repeatable and reviewable, and the other evaporates.

A skill typically holds the role, the procedure, the required output shape, and the boundaries:

```
name: env-archaeologist
description: Finds every environment variable a project reads

Procedure:
1. Search for process.env, os.environ, getenv across all source
2. Check Dockerfiles, compose files, and CI configs for declared vars
3. Compare against any existing .env.example
4. For each variable record: name, file:line, required or optional, default

Output: a markdown table, plus a list of variables used in code
but missing from .env.example.

Do not invent variables. If a value's purpose is unclear, mark it
UNKNOWN rather than guessing.
```

Note the last instruction. Every skill you write should say what to do with uncertainty, because the default behaviour is to fill the gap confidently.

Why this matters more than it appears for your submission. The challenge asks you to improve a *workflow*. A workflow is by definition something that happens more than once. A clever one-off prompt does not demonstrate a workflow improvement — it demonstrates a clever prompt. Skills are the artifact that proves repeatability, and they are the feature most entrants will skip because typing a prompt feels faster in the moment.

They are also what makes your result reproducible by a judge, which is worth real credit when the alternative is asking them to take your word for it.

MCP — connecting agents to real systems

The Model Context Protocol is a standard way for agents to talk to external tools. Bob supports it, as do most current agentic tools.

The problem it solves is combinatorial. Before a standard, every agent needed custom integration code for every system — N agents times M tools, all written by hand. MCP makes a tool expose its capabilities once, in a common format, and any MCP-aware agent can use it.

A server might expose your issue tracker, a database, a documentation system, or your CI. The agent discovers what tools are available and calls them through the same mechanism described in the Tools section — nothing new conceptually, just a shared protocol for the plumbing.

For a two-day build, treat this as knowledge rather than scope. You do not need to write an MCP server, and attempting one will eat a day you cannot spare. What you need is to understand the concept well enough to answer if asked, and to name it as a natural extension: "the doc reconciler currently reads local files; via MCP it would read the live Confluence space."

That framing is genuinely useful in a submission. It shows you know where your prototype's boundary is and what lies just past it — which reads as engineering judgement rather than as a gap.

How agents fail

Know these five before you build, because you will meet all of them and recognising a failure is most of fixing it.

FAILURE	WHAT YOU SEE	THE GUARDRAIL
Hallucination	Confident reference to a function that does not exist	Require file:line citations; verify by opening
Error compounding	Step 1 is slightly wrong, step 8 is nonsense	Checkpoints between stages; verify before proceeding
Runaway loop	Same failing action, repeatedly	Step limits; "if this fails twice, stop and report"
Silent no-op	Cheerful "done!", nothing changed	Demand evidence — the diff, the test output
Scope creep	Asked for a fix, got 14 files refactored	Explicit boundaries: "only touch <code>src/config/</code> "

Error compounding is the one that specifically threatens multi-agent pipelines. A single agent making a small mistake usually self-corrects when it sees a bad result. In a pipeline, stage 1's small error becomes stage 2's trusted input. By the end it is confident, detailed, and wrong, with no signal that anything went astray.

The defence is a checkpoint between stages: each subagent's output gets a sanity check before the next stage consumes it. For Orient, the env archaeologist's output is checkable mechanically — does `npm start` actually succeed with that `.env.example` ? A check that runs code beats a check that reads text.

Human-in-the-loop

Approval gates are where a human confirms before the agent proceeds. Put them where a mistake is expensive or hard to reverse: writing files, running migrations, anything touching git history or a remote.

The design skill is choosing *how many*. Too few and a bad run does real damage. Too many and you have rebuilt manual work with extra typing, which quietly destroys the productivity claim your whole submission depends on.

The usual answer: gate on writes and destructive actions, never on reads. Reading is cheap, reversible, and where most of the value is generated. Say this explicitly in your write-up — it is exactly the kind of deliberate trade-off judges are looking for.

Proving it works

The challenge says *clearly demonstrate impact*, and this is where most submissions quietly fail. They show a working demo and assert a benefit. Assertion is not demonstration.

Measure a baseline first

Before building anything, time the manual version. Sit with a stopwatch, or better, watch someone who has never seen the repo attempt it. Record the session.

This feels like a detour on a two-day budget. It is the highest-leverage hour you will spend, because everything you claim afterwards is anchored to a real number instead of an estimate.

Pick metrics before you see results

Choosing metrics after the fact is how you end up with numbers nobody believes, and experienced judges can smell it. Lock these in advance:

METRIC	BASELINE	WITH ORIENT
Minutes to a running local environment	measure	measure
Questions needing a human	count	count
Doc claims contradicted by code	count	count
Minutes to first correct change	measure	measure

The verification pass is your strongest evidence

"An agent with no prior context completed a starter task using only the generated pack" is a far stronger claim than "the output looks comprehensive", because it is falsifiable and you can run it live.

Make the starter task specific and checkable — something with a test that passes or fails. "Add validation to the signup endpoint so the existing test suite passes" gives a binary result. "Improve the code" gives you nothing.

Report honestly

State your sample size. If you ran one baseline with one person, say so. A judge who spots an inflated claim discounts everything else you said, and the downside is much larger than the upside.

Include what your system does not do. Name where it would break — a monorepo, an unfamiliar language, a project with no tests. **A stated limitation reads as rigour and costs you nothing.** An overclaim a judge catches costs you the submission.

Day 2 exercise — design the pipeline

Do this on paper, before touching Bob. Ninety minutes. The output is a one-page design you can hand to a judge.

1. Write the subagent roster

For each of the five agents, fill in four things. Be concrete — vagueness here becomes a broken pipeline tomorrow.

AGENT	INPUT	OUTPUT (EXACT SHAPE)	FAILURE BEHAVIOUR
Scout	repo path	Layout summary, languages, entry points	—
Env archaeologist	repo + layout	Table: var, file:line, required, default	—
Dependency cartographer	repo + layout	Module graph, entry points, dead code list	—
Doc reconciler	repo + README/PDF	List of doc claims contradicted by code	—
Risk analyst	git history + tests	Files ranked by churn × low coverage	—

The failure column is the one that will feel skippable. Fill it in anyway — write what each agent does when it cannot find what it expects. "Mark UNKNOWN and continue" and "stop and report" are both valid; silently inventing is not.

2. Decide the checkpoints

Where does a human approve? Where does a mechanical check run? Write one sentence for each. Remember the rule: gate on writes, not on reads.

3. Define the starter task

Write the exact task the verifier agent will attempt, and the exact condition for success. It must be binary. If you cannot state the pass condition in one sentence, the task is too vague.

4. Lock the metrics

Fill in the baseline column of the measurement table. If you have not measured it yet, write the number you expect, then measure and note both. Being wrong about your own estimate is an interesting finding, not an embarrassment.

Then, and only then, open Bob

Start in Plan mode with this design as your input. You will get a far better plan by handing it a considered design than by asking it to invent one, and you will be able to tell when its plan is wrong — which you cannot do if you had no design of your own.

Glossary

TERM	MEANING
Agent	A model running in a loop with tools, pursuing a goal
Agent mode	Execution mode — the agent may modify files
Approval gate	A point where a human confirms before the agent proceeds
Ask mode	Read-only mode for questions about a codebase
Context window	The total text a model can consider in one request
Context rot	Accuracy decline as the window fills with material
Fan-out / fan-in	Parallel subagents whose results are merged
Function calling	The mechanism by which a model requests a tool
Hallucination	Confident output not grounded in real evidence
MCP	Model Context Protocol — a standard for connecting agents to tools
Multi-agent	A system where several agents divide a task
Orchestration	The design of how agents and steps are arranged
Plan mode	Read-only mode producing a reviewable plan
Prompt	The instruction text sent to a model
Skill	A saved, reusable, named workflow
Stateless	Retaining nothing between calls; history is re-sent each time
Subagent	A separate agent with its own context, given a narrow task
System prompt	Persistent instructions sent with every request
Token	The unit of text a model processes; roughly 3–4 characters
Tool	A function the model may request be called
Verifier	An independent agent that checks another's output

Limits of this guide, and what comes next

This guide is scoped to what you need in two days. Four things it deliberately leaves out, so you know what you do not know:

- **How models are built.** Training, fine-tuning, and model internals. Not needed to build with them.

- **RAG and vector databases.** A large adjacent topic. Agents that search a codebase directly, as yours does, do not need it.
- **Framework specifics.** LangChain, CrewAI, AutoGen and similar. Bob handles orchestration for you; learning a framework now would be a detour.
- **Cost and latency engineering.** Real concerns in production, largely irrelevant at prototype scale.

Read next, in this order

1. IBM Bob's own documentation, particularly subagents and skills — the authority on what your tool actually does: bob.ibm.com/docs/ide
2. Anthropic's engineering writing on building agents — the clearest public material on multi-agent design trade-offs
3. The MCP specification, skimmed only, for vocabulary

The one thing to carry into the build

Agents are not intelligent assistants you delegate to and trust. They are fast, tireless, literal-minded workers with no memory and no judgement about when they are wrong.

Every design decision in your pipeline — what each agent sees, what it may touch, when a human approves, how the output gets checked — follows from taking that sentence seriously. The people who build good agentic systems are not the ones with the best prompts. They are the ones who designed for the failure modes.

Sources: IBM Bob documentation (bob.ibm.com/docs/ide) · IBM Bob launch announcement, 28 April 2026 (newsroom.ibm.com)